IQAstudy 项目学习总结报告

目录

a. 项目背景与目标
b. 技术栈与工具
c. 架构与设计
d. 核心代码与算法
e. 遇到的挑战与解决方案
f. 测试与调试过程
g. 版本控制与协作
h. 学到的新技能与知识点
i. 自我评估与未来改进

---

a. 项目背景与目标

项目背景

IQAstudy 是一个**图像质量评估（Image Quality Assessment, IQA）**的学习项目，旨在通过系统性的代码实践，掌握从深度学习基础到模型部署的完整技术栈。

图像质量评估是计算机视觉领域的重要研究方向，主要任务是通过算法自动预测人类对图像质量的感知分数。IQA 技术广泛应用于：

- **图像压缩**：评估压缩算法的质量损失
- **图像增强**：评估增强后图像的画质提升
- **图像生成**：评估 GAN、扩散模型生成图像的质量
- **视频处理**：评估视频转码、传输后的质量

项目目标

本项目的学习目标包括：

1. **掌握 Python 面向对象编程** - 理解类、继承、封装等概念
2. **掌握 NumPy 与 PyTorch 基础** - 理解张量操作、GPU 加速、自动求导
3. **掌握数据处理流程** - Dataset、DataLoader、数据增强

4. **掌握模型训练与评估** - 训练循环、损失函数、评价指标
5. **掌握迁移学习技术** - 使用预训练模型（ResNet、ViT）
6. **掌握模型部署** - ONNX 导出、C++ 推理

学习路径设计

项目采用**循序渐进**的设计思路，每天一个主题，共 20 天的学习内容：

阶段	内容	天数
基础篇	OOP, NumPy, PyTorch Tensor	Day 1-3
数据处理篇	Dataset, DataLoader, Pandas, 数据增强	Day 4, 8-9
模型训练篇	CNN 定义, 训练循环, 推理, 评价指标	Day 5-7
高级技术篇	ResNet, 微调, Checkpoint, TensorBoard, 调度器, ViT	Day 10-15
ONNX 部署篇	ONNX 导出, C++ 推理, SDK 封装, 异步视频处理	Day 16-20

---

b. 技术栈与工具

编程语言

语言	用途
Python 3	主要开发语言，用于模型训练、数据处理
C++ 17	生产级推理部署

核心框架与库

Python 端

库/框架	版本要求	用途
PyTorch	最新稳定版	深度学习框架
NumPy	-	数值计算、数组操作

库/框架	版本要求	用途
Pandas	-	数据分析、CSV 处理
SciPy	-	统计计算（SRCC, PLCC）
torchvision	与 PyTorch 匹配	预训练模型、数据增强
Pillow (PIL)	-	图像处理
OpenCV	-	C++ 图像处理
ONNX	-	ONNX 模型操作
ONNX Runtime	1.16.3	C++ 推理引擎
TensorBoard	-	训练可视化

C++ 端

库	用途
ONNX Runtime C++ API	模型推理
OpenCV	图像读取与预处理
CMake	构建系统

开发工具

工具	用途
CMake	C++ 项目构建
TensorBoard	训练过程可视化
Jupyter/Python REPL	交互式开发

硬件环境

组件	说明
GPU	CUDA 可用时自动使用 GPU 加速
CPU	备用计算设备

c. 架构与设计

整体架构

```
┌──────────────────────────────────────────────────────────────────┐
│                          IQAstudy 架构                           │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────┐    │
│   │   数据层     │     │   模型层     │     │   部署层     │    │
│   ├──────────────┤     ├──────────────┤     ├──────────────┤    │
│   │              │     │              │     │              │    │
│   │ • CSV读取    │     │ • SimpleCNN  │     │ • ONNX导出   │    │
│   │ • Dataset    │     │ • ResNet18   │     │ • C++推理    │    │
│   │ • Transform  │     │ • ViT-B/16   │     │ • OpenCV     │    │
│   │ • DataLoader │     │              │     │              │    │
│   │              │     │              │     │              │    │
│   └──────────────┘     └──────────────┘     └──────────────┘    │
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                        训练流程                          │   │
│   │  DataLoader → Model → Loss → Backprop → Optimizer        │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

模块设计

1. 数据模块（Day 4, 8-9）

```python
# Day 4: 基础 Dataset
class IQADataset(Dataset):
    def __init__(self, txt_file):
        self.data_info = []  # 存储 [文件名, 分数]

    def __len__(self):
        return len(self.data_info)

    def __getitem__(self, idx):
        filename, score = self.data_info[idx]
        fake_image = torch.randn(3, 224, 224)
```

```
        label = torch.tensor(score, dtype=torch.float32)
        return fake_image, label
```

```
# Day 9: 高级 Dataset（集成 Pandas + Transform）
class ProfessionalDataset(Dataset):
    def __init__(self, csv_file, transform=None):
        self.df = pd.read_csv(csv_file)
        self.transform = transform

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.fromarray(...)
        if self.transform:
            img_tensor = self.transform(img)
        return img_tensor, label
```

2. 模型模块（Day 5, 10, 15）

```
# Day 5: 自定义 CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        self.conv1 = nn.Conv2d(3, 16, 3)
        self.relu = nn.ReLU()
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(16, 1)

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

```
# Day 10: 迁移学习 ResNet
class ResNetIQA(nn.Module):
    def __init__(self):
        self.backbone = models.resnet18(weights='DEFAULT')
        self.backbone.fc = nn.Linear(512, 1)

    def forward(self, x):
        return self.backbone(x)
```

3. 训练模块（Day 5, 12-14）

训练流程遵循标准深度学习范式：

```
# 标准训练循环
for epoch in range(num_epochs):
    model.train()
    for batch_idx, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels.view(-1, 1))
        loss.backward()
        optimizer.step()

    # 验证阶段
    model.eval()
    with torch.no_grad():
        for images, labels in val_loader:
            ...
```

4. 部署模块（Day 16-18）

```
PyTorch Model (.pth)
        |
        ▼ torch.onnx.export
ONNX Model (.onnx)
        |
        ▼ ONNX Runtime (C++)
C++ Inference Program
```

数据流设计

```
输入图片 (HWC, BGR, 0-255)
        |
        ▼ OpenCV/PIL 读取
cv::Mat / PIL Image
        |
        ▼ Resize (224x224)
224x224 Image
```

```
        |
        ▼ BGR→RGB, Normalize
CHW, RGB, [-1, 1]
        |
        ▼ ToTensor
Tensor [1, 3, 224, 224]
        |
        ▼ Model Inference
Score [1, 1]
```

---

d. 核心代码与算法

核心算法

1. 图像质量评价指标（Day 7）

IQA 任务使用两个核心指标：

SRCC (Spearman Rank Correlation Coefficient)

- 中文名：Spearman 等级相关系数
- 作用：衡量预测分数与真实分数的**排名一致性**
- 特点：对分数的线性偏移不敏感
- 公式：$\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$
- 理想值：1.0（完全正相关）

PLCC (Pearson Linear Correlation Coefficient)

- 中文名：Pearson 线性相关系数
- 作用：衡量预测分数与真实分数的**线性拟合度**
- 特点：对分数偏移敏感
- 公式：$r = \frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum(x_i-\bar{x})^2}\sqrt{\sum(y_i-\bar{y})^2}}$
- 理想值：1.0（完美线性关系）

```python
def compute_metrics(pred, target):
    srcc, _ = stats.spearmanr(pred, target)
```

```
    plcc, _ = stats.pearsonr(pred, target)
    return srcc, plcc
```

2. 数据增强流水线（Day 9）

```
train_transform = transforms.Compose([
    transforms.Resize((256,256)),           # 调整大小
    transforms.RandomCrop((224,224)),       # 随机裁剪
    transforms.RandomHorizontalFlip(p=0.5), # 随机翻转
    transforms.ToTensor(),                   # 转 Tensor
    transforms.Normalize(                    # 标准化
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

ImageNet 标准化参数是业界通用标准，使用该参数可以利用预训练模型的特征分布。

3. 迁移学习与微调（Day 10-11）

迁移学习流程：

1. 加载 ImageNet 预训练权重
2. 修改最后一层全连接层（Head）
3. 可选：冻结特征提取器参数

```
# Day 10: 修改 Head
resnet = models.resnet18(weights='DEFAULT')
resnet.fc = nn.Linear(512, 1)  # 1000→1

# Day 11: 冻结参数
for para in model.parameters():
    para.requires_grad = False
model.fc = nn.Linear(512, 1)  # 新层默认 requires_grad=True
```

4. 学习率调度（Day 14）

使用 `StepLR` 实现阶梯式学习率衰减：

```python
scheduler = optim.lr_scheduler.StepLR(
    optimizer,
    step_size=10,  # 每10个epoch
    gamma=0.1      # 学习率乘以0.1
)

for epoch in range(30):
    # 训练...
    scheduler.step()  # epoch结束时调用
```

5. ONNX 导出与动态轴（Day 16）

```python
torch.onnx.export(
    model,
    dummy_input,
    "resnet_iqa.onnx",
    export_params=True,
    opset_version=11,
    input_names=['input'],
    output_names=['output'],
    dynamic_axes={
        'input': {0: 'batch_size'},
        'output': {0: 'batch_size'}
    }
)
```

`dynamic_axes` 设置允许推理时使用任意 batch size。

6. C++ 图像预处理（Day 18）

```cpp
std::vector<float> Preprocess(cv::Mat& img) {
    cv::resize(img, img, cv::Size(224, 224));
    cv::cvtColor(img, img, cv::COLOR_BGR2RGB);

    float mean[] = {0.485, 0.456, 0.406};
    float std[] = {0.229, 0.224, 0.225};

    std::vector<float> tensor;
    for (int h = 0; h < 224; h++) {
        for (int w = 0; w < 224; w++) {
            for (int c = 0; c < 3; c++) {
                float pixel = (img.at<cv::Vec3b>(h, w)[c] / 255.0f - mean[c])
```

```
/ std[c];
                tensor[c * 224 * 224 + h * 224 + w] = pixel;
            }
        }
    }
    return tensor;
}
```

7. ONNX SDK 封装与 CUDA 加速（Day 19）

封装设计：
将 ONNX Runtime 的初始化、预处理、推理封装成 `IQAModel` 类，实现极简调用：

```
class IQAModel {
public:
    IQAModel(const std::string& model_path);  // 加载模型
    float Predict(cv::Mat& img);               // 一行预测

private:
    std::vector<float> Preprocess(cv::Mat& img);
    Ort::Env env;
    Ort::SessionOptions session_options;
    std::unique_ptr<Ort::Session> session;
};
```

CUDA 硬件加速：

```
IQAModel::IQAModel(const std::string& model_path) {
    // 启用 CUDA 加速
    OrtCUDAProviderOptions cuda_options;
    cuda_options.device_id = 0;
    session_options.AppendExecutionProvider_CUDA(cuda_options);
    session_options.SetGraphOptimizationLevel(ORT_ENABLE_ALL);

    // 加载模型
    session = std::make_unique<Ort::Session>(env, model_path.c_str(),
session_options);
}
```

使用示例：

```
IQAModel model("resnet_iqa.onnx");  // 初始化
float score = model.Predict(img);    // 预测
```

8. 异步生产者-消费者模型（Day 20）

架构设计：

```
┌──────────────┐          ┌──────────────┐
│  生产者线程   │          │  消费者线程   │
│ (VideoReader)│ ────────► │  (GPU推理)   │
│   CPU I/O    │  Queue   │   高延迟      │
└──────────────┘          └──────────────┘
```

核心实现：

```
// 全局队列与同步原语
std::queue<cv::Mat> frame_queue;
std::mutex mtx;
std::condition_variable cv_queue;
std::atomic<bool> is_reading_finished(false);

// 生产者线程
void VideoReaderThread(const std::string& video_path) {
    cv::VideoCapture cap(video_path);
    cv::Mat frame;
    while (cap.read(frame)) {
        std::unique_lock<std::mutex> lock(mtx);
        cv_queue.wait(lock, []{ return frame_queue.size() < 30; });
        frame_queue.push(frame.clone());
        lock.unlock();
        cv_queue.notify_one();
    }
    is_reading_finished = true;
    cv_queue.notify_all();
}

// 消费者线程（主线程）
while (true) {
    std::unique_lock<std::mutex> lock(mtx);
    cv_queue.wait(lock, []{ return !frame_queue.empty() ||
is_reading_finished; });
```

```
    if (frame_queue.empty() && is_reading_finished) break;
    frame = frame_queue.front();
    frame_queue.pop();
    lock.unlock();

    float score = model.Predict(frame);  // GPU 推理
}
```

分数平滑滤波：

```
std::deque<float> score_buffer;
const int SMOOTH_WINDOW = 10;

score_buffer.push_back(raw_score);
if (score_buffer.size() > SMOOTH_WINDOW) score_buffer.pop_front();
float smooth_score = accumulate(score_buffer.begin(), score_buffer.end(), 0.0)
/ score_buffer.size();
```

ONNX 部署完整流程

```
┌──────────────────────────────────────────────────────────────────────┐
│                        ONNX 部署完整流程                              │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Day 16: PyTorch 模型导出                                            │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ torch.onnx.export(model, dummy_input, "model.onnx")          │   │
│  │ • 设置 opset_version=11                                      │   │
│  │ • 配置 dynamic_axes 支持动态 batch                           │   │
│  │ • onnx.checker.check_model() 验证                            │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                  ↓                                   │
│  Day 17-18: C++ ONNX Runtime 基础推理                                │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ • Ort::Env + Ort::Session 初始化                             │   │
│  │ • Ort::Value::CreateTensor<float>() 创建输入                 │   │
│  │ • session.Run() 执行推理                                     │   │
│  │ • 图像预处理：Resize → BGR2RGB → Normalize → HWC2CHW        │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                  ↓                                   │
│  Day 19: SDK 封装 + CUDA 加速                                        │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ • IQAModel 类封装                                            │   │
```

```
│   │ • OrtCUDAProviderOptions GPU 加速                          │   │
│   │ • GraphOptimizationLevel 图优化                            │   │
│   │ • unique_ptr 资源管理                                      │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                ↓                                   │
│   Day 20: 异步高并发视频处理                                       │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │ • 生产者-消费者模型                                         │   │
│   │ • std::thread 多线程                                       │   │
│   │ • mutex + condition_variable 同步                          │   │
│   │ • 分数平滑滤波 UI 优化                                      │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

ONNX 部署关键参数

参数	说明	推荐值
`opset_version`	ONNX 算子版本	11
`dynamic_axes`	动态维度支持	`{0: 'batch_size'}`
`device_id`	GPU 设备 ID	0
`GraphOptimizationLevel`	图优化级别	`ORT_ENABLE_ALL`
队列深度	视频处理缓冲	30 帧
平滑窗口	分数平滑	10 帧

---

e. 遇到的挑战与解决方案

挑战 1：NumPy 与 PyTorch 张量维度转换

问题：图像数据从 HWC 格式转换为 CHW 格式时容易混淆。

解决方案：

- HWC（高度-宽度-通道）：图片原始格式
- CHW（通道-高度-宽度）：深度学习框架格式
- 使用 `transpose(2, 0, 1)` 进行转换

挑战 2：GPU 与 CPU 设备兼容

问题：在 GPU 上训练的模型无法直接在 CPU 上加载。

解决方案：

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.load_state_dict(torch.load("model.pth", map_location=device))
```

挑战 3：训练时梯度累加

问题：如果不清零梯度，梯度会累加导致训练错误。

解决方案：

```
optimizer.zero_grad()  # 每个 batch 前清零梯度
```

挑战 4：模型.eval() 与训练模式混淆

问题：推理时忘记切换到 eval 模式，导致 BatchNorm 和 Dropout 行为异常。

解决方案：

```
model.eval()  # 推理前切换
with torch.no_grad():
    output = model(input)
```

挑战 5：ONNX 动态 batch 支持

问题：导出的 ONNX 模型默认只支持固定 batch size。

解决方案：设置 `dynamic_axes` 参数：

```
dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}}
```

挑战 6：C++ 图像格式转换

问题：OpenCV 读取的图像是 HWC BGR 格式，需要转换为 CHW RGB 格式。

解决方案：手动实现像素重排（Day 18 的 Preprocess 函数）

挑战 7：学习率设置不当

问题：学习率过大导致 loss 不收敛，学习率过小导致训练太慢。

解决方案：使用学习率调度器（Day 14），初始设高学习率，后期衰减。

挑战 8：CUDA 加速配置（Day 19）

问题：ONNX Runtime 默认使用 CPU 推理，无法利用 GPU 加速。

解决方案：配置 CUDA Provider：

```
OrtCUDAProviderOptions cuda_options;
cuda_options.device_id = 0;
session_options.AppendExecutionProvider_CUDA(cuda_options);
session_options.SetGraphOptimizationLevel(ORT_ENABLE_ALL);
```

挑战 9：视频处理内存溢出（Day 20）

问题：同步读取视频时，帧积压导致内存爆炸。

解决方案：使用生产者-消费者模型，限制队列深度：

```
cv_queue.wait(lock, []{ return frame_queue.size() < 30; });
```

挑战 10：多线程数据竞争（Day 20）

问题：生产者和消费者同时访问队列导致数据竞争。

解决方案：使用互斥锁和条件变量：

```
std::mutex mtx;
std::condition_variable cv_queue;
std::unique_lock<std::mutex> lock(mtx);
```

---

f. 测试与调试过程

测试策略

测试类型	方法	覆盖内容
单元测试	运行每个 .py 文件	数据处理、模型定义
集成测试	完整训练流程	Day 5, Day 12
推理测试	加载模型预测	Day 6, Day 16-18
可视化测试	TensorBoard	训练曲线、学习率变化

调试技巧

1. 打印形状检查

```
print(f"Input shape: {images.shape}")  # 预期 [B, 3, 224, 224]
print(f"Output shape: {output.shape}")  # 预期 [B, 1]
```

2. 设备检查

```
print(f"Model device: {next(model.parameters()).device}")
print(f"Input device: {images.device}")
```

3. 梯度检查

```
print(f"Requires grad: {x.requires_grad}")
print(f"Gradient: {x.grad}")
```

4. ONNX 模型验证

```
onnx_model = onnx.load("model.onnx")
onnx.checker.check_model(onnx_model)
```

常见错误排查

错误	原因	解决方案
CUDA out of memory	batch size 过大	减小 batch size
RuntimeError: expected dtype=float32	数据类型不匹配	使用 `dtype=torch.float32`
模型输出 NaN	学习率过大或数据未归一化	调小学习率，检查 Normalize
ONNX export failed	模型有动态操作	避免动态操作或设置 dynamic_axes

---

g. 版本控制与协作

文件结构

```
IQAstudy/
├── day1_oop.py ~ day18_image.cpp  # 每日代码
├── CMakeLists.txt                   # 构建配置
├── resnet_iqa.onnx                  # 导出的模型
├── run/                             # TensorBoard 日志
│   └── iqa_experimnet/
├── build/                           # CMake 构建目录
└── lib/                            # ONNX Runtime 库
```

关键产物

文件	说明
`iqa_model.pth`	Day 5 训练保存的模型
`best_model.pth`	Day 12 最佳验证模型
`resnet_iqa.onnx`	Day 16 导出的 ONNX 模型
`run/iqa_experimnet/`	TensorBoard 日志

版本管理建议

1. **代码版本**：使用 Git 管理代码版本
2. **模型版本**：记录每个模型的训练配置和性能
3. **数据版本**：记录数据集的版本和统计信息

---

h. 学到的新技能与知识点

编程技能

技能	掌握程度	应用场景
Python OOP	✓ 熟练	代码架构、模块化
NumPy 多维数组	✓ 熟练	数据预处理
PyTorch Tensor	✓ 熟练	深度学习开发
C++ 基础	✓ 了解	生产部署
CMake 构建	✓ 了解	C++ 项目编译

深度学习技能

技能	掌握程度	应用场景
自定义 Dataset	✓ 熟练	数据加载
数据增强	✓ 熟练	提升模型泛化
CNN 设计	✓ 了解	模型架构
迁移学习	✓ 熟练	快速上手新任务
模型微调	✓ 熟练	特定任务优化
学习率调度	✓ 了解	训练优化
ONNX 导出	✓ 熟练	跨框架部署

部署技能

技能	掌握程度	应用场景
ONNX Runtime (C++)	✓ 熟练	高性能推理
OpenCV 图像处理	✓ 熟练	图像预处理
TensorBoard	✓ 熟练	训练监控
CUDA Provider	✓ 熟练	GPU 加速推理
SDK 封装	✓ 熟练	代码工程化
C++ 多线程	✓ 了解	异步处理
生产者-消费者模型	✓ 了解	高并发架构

评价指标理解

指标	理解
SRCC	排名相关性，对偏移不敏感
PLCC	线性相关性，对偏移敏感

---

i. 自我评估与未来改进

自我评估

掌握程度评分（5分制）

模块	评分	说明
Python 基础	4.5	OOP、NumPy 熟练掌握
PyTorch 基础	4.5	Tensor、Dataset、训练流程熟练
数据处理	4.0	Pandas、数据增强掌握
模型训练	4.0	训练循环、checkpoint 掌握
迁移学习	4.5	ResNet、ViT、微调掌握
ONNX 部署	4.5	导出、C++推理、SDK封装、异步处理掌握
可视化	4.0	TensorBoard 掌握
C++ 多线程	3.5	生产者-消费者模型了解

总体评价

优点：

- 学习路径清晰，循序渐进
- 每个主题都有完整的代码示例
- 覆盖了从训练到部署的完整流程
- ONNX 部署从基础到生产级架构全覆盖
- 包含 CUDA 加速和异步高并发处理

待加强：

- C++ 多线程编程可以更深入
- 缺少实际的 IQA 数据集处理
- 没有涉及模型调参的系统性方法
- 视频处理可以添加更多优化策略

未来改进方向

1. 深入模型架构

- ◯ 实现更复杂的 IQA 模型（如 MANIQA、NIQE）
- ◯ 学习 Transformer 架构的深入细节
- ◯ 探索 GAN 在 IQA 中的应用

2. 完善数据处理

- ◯ 使用真实的 IQA 数据集（如 LIVE、CSIQ、KONIQ-10k）
- ◯ 实现更复杂的数据增强策略
- ◯ 学习数据清洗和异常值处理

3. 优化训练流程

- ◯ 实现 Early Stopping
- ◯ 添加 Mixed Precision Training（混合精度）
- ◯ 实现分布式训练
- ◯ 学习超参数自动搜索

4. 提升部署能力

- [ ] 学习 TensorRT 部署（更快的推理速度）
- [ ] 实现 Web 服务部署（Flask/FastAPI）
- [ ] 探索移动端部署（ONNX Mobile、TensorFlow Lite）
- [ ] 实现模型量化（INT8/FP16）
- [ ] 添加批处理推理优化
- [ ] 实现多路视频流并发处理

5. 增加测试覆盖

- [ ] 添加单元测试
- [ ] 实现数据一致性验证
- [ ] 添加性能基准测试

扩展学习资源

资源	内容
PyTorch Documentation	官方 API 文档
ONNX Runtime Documentation	部署相关
论文：NIQE, BRISQUE, LIQE	IQA 经典算法
Hugging Face Transformers	ViT 进阶

总结

IQAstudy 项目通过 **20 天**的系统学习，成功覆盖了**从 Python 基础到深度学习部署**的完整技术栈。通过本项目：

1. **掌握了 PyTorch 深度学习开发能力**
2. **理解了迁移学习和模型微调技术**
3. **学会了模型导出和跨平台部署**
4. **了解了 IQA 领域的核心评价指标**
5. **掌握了 ONNX SDK 封装和 CUDA 加速**
6. **实现了生产级异步视频流处理架构**

ONNX 部署里程碑

阶段	天数	核心能力
模型导出	Day 16	PyTorch → ONNX，动态轴支持
基础推理	Day 17-18	C++ ONNX Runtime，图像预处理
SDK 封装	Day 19	类封装，CUDA 加速，工程化
高并发处理	Day 20	生产者-消费者模型，视频流处理

这个学习路径为进一步深入图像质量评估研究奠定了坚实的基础，也为未来的工程实践提供了宝贵的技术储备。

报告生成时间：2026年2月7日
项目路径：/workspace/examples/IQAstudy